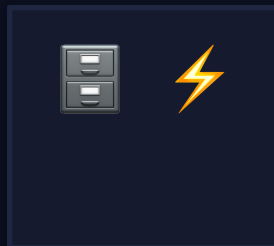


```
SELECT * FROM users;
```



# ORM & MySQL

From Raw SQL to Object-Relational Mapping 🚀

>\_ TA Ahmed Mohamed & TA. Ahmed Mostafa | Section 3

```
session.add(user)
```

# What We'll Cover Today



## MySQL Fundamentals

Tables, Data Types, Keys, Joins



## Relationships

One-to-Many, Many-to-Many



## What is an ORM?

Mapping Python classes to DB tables



## SQLAlchemy Setup

Engine, Session, Base



## CRUD with ORM

Create · Read · Update · Delete



## ORM vs Raw SQL

When to use each approach

# What is a Database?



## Organized Storage

for your application data



## Structured Storage

Data is organized into tables with rows & columns — just like a spreadsheet, but much more powerful.



## Queryable

You can search, filter, sort and combine data using SQL (Structured Query Language).



## Persistent

Data survives even when your app restarts — unlike Python variables or lists in memory.

# Why MySQL?



## Most Popular

MySQL is the world's most widely-used open-source relational database. Used by Facebook, YouTube, and Twitter.



## Free & Open Source

No license fees. Easy to install on any platform — Windows, Mac, or Linux.



## Fast & Reliable

Optimised for read-heavy workloads. Handles millions of rows efficiently with the right indexes.



## Great Python Support

Works seamlessly with SQLAlchemy, PyMySQL, mysql-connector-python and FastAPI.



Note: MySQL uses a relational model — data lives in tables that can relate to each other via Foreign Keys.

# MySQL Core Concepts



## Table

A structured grid that stores data of the same type (e.g. Users, Products).



## Row

A single record in a table — one user, one product, one order.



## Column

An attribute/field that every row must have (e.g. name, email, age).



## Primary Key

A unique identifier for every row — no duplicates allowed (e.g. id).



### users table

| id | name  | email          | age |
|----|-------|----------------|-----|
| 1  | Alice | alice@mail.com | 24  |
| 2  | Bob   | bob@mail.com   | 30  |
| 3  | Carol | carol@mail.com | 22  |
| 4  | Dave  | dave@mail.com  | 28  |

# MySQL Data Types

## Numeric

- INT — whole numbers (1, 42, -7)
- FLOAT / DOUBLE — decimals
- DECIMAL(10,2) — precise money values

## String

- VARCHAR(n) — variable-length text (most common)
- TEXT — long paragraphs or descriptions
- CHAR(n) — fixed-length string

## Date & Time

- DATE — YYYY-MM-DD
- DATETIME — YYYY-MM-DD HH:MM:SS
- TIMESTAMP — auto-updated timestamps

## Other

- BOOLEAN / TINYINT(1) — True/False
- ENUM('a','b') — fixed set of values
- JSON — store JSON objects

# Creating Tables in MySQL

## CREATE TABLE

Defines the structure — columns & their data types.

## PRIMARY KEY

Each row gets a unique auto-incrementing id.

## NOT NULL

Forces every row to have a value for this column.

## VARCHAR(100)

Text field that holds up to 100 characters.

## SQL

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL UNIQUE,  
  age INT,  
  created TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

✓ Best Practice: Always add an id column as AUTO\_INCREMENT PRIMARY KEY.

# CRUD Operations in SQL

## C INSERT

```
INSERT INTO users (name, email)
VALUES ('Alice', 'alice@mail.com');
```

## R SELECT

```
SELECT * FROM users
WHERE age > 20
ORDER BY name;
```

## U UPDATE

```
UPDATE users
SET email = 'new@mail.com'
WHERE id = 1;
```

## D DELETE

```
DELETE FROM users
WHERE id = 1;
```

# Primary Keys & Foreign Keys



## Primary Key (PK)

- Uniquely identifies every row in a table
- Cannot be NULL — every row MUST have one
- Usually an auto-incrementing integer id
- One table can only have ONE primary key



## Foreign Key (FK)

- References the PK of another table
- Creates a RELATIONSHIP between tables
- Many rows can reference the same parent

### SQL

```
CREATE TABLE posts (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(200) NOT NULL,  
  user_id INT,  
  FOREIGN KEY (user_id) REFERENCES users(id)  
);
```

# Table Relationships

## 1 One-to-One

One record links to exactly one other record.  
Example: User  $\leftrightarrow$  UserProfile  
Use: FOREIGN KEY with UNIQUE constraint.

## 1 $\rightarrow$ $\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}$ One-to-Many

One record relates to many others.  
Example: User  $\rightarrow$  Posts (one user writes many posts)  
Use: Foreign key on the 'many' side table.

## $\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}$ $\leftrightarrow$ $\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}$ Many-to-Many

Many records on both sides.  
Example: Students  $\leftrightarrow$  Courses  
Use: A junction/bridge table in the middle.



Most web apps use One-to-Many heavily. Always put the FK on the 'child' table side.

# SQL JOINS – Combining Tables

## INNER JOIN

Returns only rows that have a match in BOTH tables.  
Most commonly used join.

## LEFT JOIN

Returns all rows from the LEFT table + matched rows from right.  
Unmatched right rows become NULL.

## RIGHT JOIN

Returns all rows from the RIGHT table + matched left rows.  
Rarely used — LEFT JOIN preferred.

## SQL

```
-- Get all posts WITH their author name
SELECT posts.title, users.name
FROM posts
INNER JOIN users
    ON posts.user_id = users.id;
```

```
-- Include posts even if no author
SELECT posts.title, users.name
FROM posts
LEFT JOIN users
    ON posts.user_id = users.id;
```

# What is an ORM?

## ORM = Object-Relational Mapper

A library that translates between Python objects and database rows — no raw SQL needed.

### Python Class

```
class User:  
    id: int  
    name: str  
    email: str
```

You write Python



### ORM Magic (SQLAlchemy)

⚙ Translates

Python → SQL  
SQL → Python

Handles mapping



### MySQL Table

```
users  
  id INT PK  
  name VARCHAR  
  email VARCHAR
```

Database stores

# Why Use an ORM?



## Write Python, Not SQL

Define tables as Python classes. Query using Python methods — feels natural and readable.



## SQL Injection Protection

ORM automatically parameterizes queries — protecting your app from malicious input.



## Database Portability

Switch from MySQL to PostgreSQL with almost zero code change. ORM abstracts the dialect.



## Less Boilerplate

No manually building SQL strings. ORM generates INSERT, UPDATE, DELETE automatically.



## Built-in Relationships

Load related objects easily: `user.posts` returns all posts for that user automatically.



## Validation & Types

Python type hints + SQLAlchemy columns ensure data integrity before it hits the DB.

# SQLAlchemy – The Python ORM



## Most Popular Python ORM

Used in Flask, FastAPI, and standalone scripts worldwide.



## Two APIs

Core (SQL expression layer) & ORM (Python object layer). We use ORM style.



## MySQL Driver

Needs PyMySQL or mysqlclient as a connector between SQLAlchemy and MySQL.



## Installation

### BASH

```
# Install SQLAlchemy  
pip install sqlalchemy
```

```
# Install MySQL connector (choose one)  
pip install pymysql  
# OR  
pip install mysqlclient
```

```
# For FastAPI integration  
pip install sqlalchemy[asyncio]
```

# SQLAlchemy Setup – database.py

## SQL

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, DeclarativeBase

# 1. Connection URL
DATABASE_URL = (
    "mysql+pymysql://root:password@localhost:3306/mydb"
)

# 2. Create Engine (one per app)
engine = create_engine(DATABASE_URL, echo=True)

# 3. Session Factory
SessionLocal = sessionmaker(
    autocommit=False, autoflush=False, bind=engine
)

# 4. Base class for all ORM models
class Base(DeclarativeBase):
    pass
```

## engine

The connection to the DB. Like a database driver.

## SessionLocal

Factory to create DB sessions for each request.

## Base

Parent class that all your models will inherit from.

## echo=True

Prints generated SQL — great for learning & debugging.

# Defining ORM Models – models.py

## SQL

```
from sqlalchemy import Column, Integer, String, ForeignKey
from sqlalchemy.orm import relationship
from database import Base
```

```
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)
    email = Column(String(150), unique=True, nullable=False)

    # Relationship to Post
    posts = relationship('Post', back_populates='author')
```

```
class Post(Base):
    __tablename__ = "posts"

    id = Column(Integer, primary_key=True, index=True)
    title = Column(String(200), nullable=False)
    user_id = Column(Integer, ForeignKey('users.id'))

    author = relationship('User', back_populates='posts')
```

## `__tablename__`

Maps the class to a real table name in MySQL.

## `Column()`

Defines a column — type, constraints, index.

## `ForeignKey()`

Creates a link to the primary key of another table.

## `relationship()`

Allows `user.posts` and `post.author` in Python — no JOIN needed.

## `Base`

All models inherit from `Base` so SQLAlchemy knows about them.

# Creating Tables & DB Session

## SQL

```
# main.py or database.py
from database import Base, engine
import models # import so Base knows them

# Create all tables in MySQL
Base.metadata.create_all(bind=engine)

# -----
# Dependency for FastAPI routes
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

## 1 Import Base + models

SQLAlchemy scans all model classes.

## 2 create\_all()

Runs CREATE TABLE SQL for every model.

## 3 get\_db() dependency

FastAPI calls this before each route.

## 4 yield db

Passes the session to the route function.

## 5 db.close()

Releases the connection back to the pool.

# ORM CRUD – Create & Read

## ✓ CREATE — Add a new record

PY

```
from models import User

def create_user(db, name: str, email: str):
    new_user = User(name=name, email=email)
    db.add(new_user) # stage the insert
    db.commit()      # write to DB
    db.refresh(new_user) # reload from DB
    return new_user
```



db.refresh(obj) reloads the object from DB so you get the auto-generated id



## READ — Query records

PY

```
# Get all users
users = db.query(User).all()

# Get by primary key
user = db.query(User).get(1)

# Filter
user = db.query(User)\
    .filter(User.email == email)\
    .first()

# Multiple filters
results = db.query(User)\
    .filter(User.age > 18)\
    .order_by(User.name)\
    .limit(10)\
    .all()
```

# ORM CRUD – Update & Delete



## UPDATE — Modify a record

PY

```
def update_user(db, user_id: int, new_name: str):  
    # 1. Fetch the record  
    user = db.query(User).get(user_id)  
  
    if not user:  
        return None  
  
    # 2. Modify the Python object  
    user.name = new_name  
  
    # 3. Commit — SQLAlchemy detects the change  
    db.commit()  
    db.refresh(user)  
    return user
```



## DELETE — Remove a record

PY

```
def delete_user(db, user_id: int):  
    user = db.query(User).get(user_id)  
  
    if not user:  
        return False  
  
    db.delete(user)  
    db.commit()  
    return True  
  
# — Bulk delete with filter —  
db.query(User)\  
    .filter(User.active == False)\  
    .delete()  
db.commit()
```



Always call `db.commit()` after add/delete/update. Without it, changes are NOT saved to the database.

# ORM vs Raw SQL – When to Use What?

## ✓ Use ORM When...

- ✓ Building standard CRUD apps & REST APIs
- ✓ Team mixes Python devs with limited SQL knowledge
- ✓ You need database portability (MySQL → PostgreSQL)
- ✓ Rapid development & clean, readable code
- ✓ Working with relationships between tables

## ⚡ Use Raw SQL When...

- ⚡ Complex analytical queries with multiple GROUP BYs
- ⚡ Performance-critical bulk operations
- ⚡ Database-specific features (stored procedures)
- ⚡ Large data migrations & batch inserts
- ⚡ Data warehousing / reporting workloads

🏆 Verdict: Start with ORM (SQLAlchemy). Drop down to raw SQL only for the 10% of cases where performance demands it.